WAZUH SECURITY VALIDATION STANDARD OPERATING PROCEDURE (SOP)

Environment: HIPAA-Compliant Infrastructure
Version: 1.0
Document Owner: Harshit Jaswal

1. Purpose

This Standard Operating Procedure (SOP) establishes a structured and repeatable methodology for validating the security monitoring capabilities of the **Wazuh Security Information and Event Management (SIEM) platform** within a HIPAA-compliant environment processing electronic Protected Health Information (ePHI).

The purpose of this validation is to ensure that Wazuh is:

- Properly ingesting and normalizing security telemetry
- Accurately detecting high-risk behaviors aligned to adversarial techniques
- Generating timely, actionable alerts
- Preserving audit integrity
- Supporting regulatory compliance evidence requirements

This SOP does **not** constitute a penetration test. Instead, it is a controlled detection engineering validation designed to measure monitoring effectiveness, alert fidelity, and compliance assurance within authorized lab or staging environments.

1.1 Validation Objectives

The validation activities defined in this document are intended to measure and confirm the following control capabilities:

Logging & Visibility

- Log ingestion completeness across Windows, Linux, database, network, and cloud systems
- Integrity and preservation of audit logs
- Time synchronization and event timestamp accuracy

Detection Engineering

- Rule accuracy and appropriate severity classification
- Correlation logic effectiveness
- False positive and false negative rates
- MITRE ATT&CK technique mapping coverage

Threat Monitoring Controls

- File Integrity Monitoring (FIM) coverage of ePHI directories
- Authentication monitoring and brute-force detection
- Privilege escalation and administrative group change detection
- Log tampering and audit impairment detection

- Suspicious process execution detection
- Data exfiltration anomaly detection
- Agent tampering and heartbeat monitoring

Performance & Response Metrics

- Alert latency (Mean Time to Detect – MTTD)
- Alert enrichment completeness (user, host, source IP, process, file path)
- Dashboard visibility and correlation integrity

Compliance Assurance

- Alignment with HIPAA Security Rule audit control requirements
- Evidence generation capability for compliance audits
- Documentation integrity and change control validation

1.2 Success Criteria

This validation is considered successful when:

1. All simulated adversarial behaviors generate alerts aligned with expected severity.
2. Alerts contain complete and accurate contextual metadata.
3. Detection occurs within defined time thresholds.
4. No production data integrity is compromised.
5. All test artifacts are fully cleaned and documented.
6. Findings are translated into measurable detection improvement actions.

1.3 Governance and Risk Boundaries

All simulations performed under this SOP must:

- Be executed in authorized and controlled environments.
- Avoid modification or exposure of live ePHI.
- Avoid destructive system actions unless explicitly approved.
- Preserve the operational integrity of Wazuh components.
- Be documented for audit traceability.

2. Scope

This SOP applies to all in-scope systems that store, process, transmit, or provide security telemetry related to electronic Protected Health Information (ePHI) within the defined HIPAA-aligned environment.

The scope includes validation of detection, logging, correlation, and alerting capabilities across infrastructure, endpoints, applications, databases, and supporting security systems monitored by Wazuh.

2.1 In-Scope System Categories

Identity & Access Infrastructure

- Windows Domain Controllers
- Active Directory Federation Services (if applicable)
- Privileged access management systems

Validation Focus:
Authentication events, group membership changes, account lockouts, privilege escalation activity, and log integrity.

Endpoint Systems

- Clinical workstation endpoints
- Administrative endpoints
- Remote access endpoints (VPN-connected systems)

Validation Focus:
Process creation events, suspicious execution, scheduled task creation, file access to monitored directories, authentication attempts, and agent health.

Application & ePHI Processing Systems

- EHR application servers
- Application middleware servers
- API gateways

Validation Focus:
Application log ingestion, privileged access monitoring, file integrity monitoring (where applicable), and anomaly detection.

Database Systems

- Microsoft SQL Server instances hosting PHI databases
- Supporting database servers in failover or high-availability configurations

Validation Focus:
Database audit logging ingestion, privileged query monitoring (if enabled), log forwarding integrity, and anomaly detection.

Linux Infrastructure

- Web servers
- API servers
- Supporting backend Linux services

Validation Focus:
auditd ingestion, SSH authentication monitoring, FIM coverage, suspicious command execution, and log tampering detection.

Network & Perimeter Systems

- VPN gateways
- Firewall log sources (if integrated)
- Network flow telemetry sources (if configured)

Validation Focus:
Authentication attempts, remote access patterns, outbound anomaly detection, and log ingestion completeness.

Backup & Recovery Systems

- Backup servers
- Replication services
- Disaster recovery nodes

Validation Focus:
Administrative access monitoring, backup integrity alerts, and FIM coverage where applicable.

Cloud & Storage Environments

- Cloud object storage repositories
- Cloud-hosted compute instances monitored by Wazuh

- Integrated cloud audit log sources

Validation Focus:
Access logging, configuration change monitoring, file/object integrity events, and API activity logging.

Wazuh Platform Components

- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard
- All deployed Wazuh Agents

Validation Focus:
Agent heartbeat monitoring, rule execution integrity, correlation logic, alert indexing, dashboard visibility, and cluster health.

2.2 Environment Boundaries

Validation activities must be executed in:

- Authorized lab environments
- Staging environments mirroring production controls
- Controlled validation windows in production (if formally approved)

Production environments handling live ePHI may only be included when:

- Change control approval is documented
- Risk assessment is completed
- Simulations are non-destructive and reversible
- Security Operations leadership approval is obtained

2.3 Telemetry Requirements for Inclusion

For a system to be considered in scope for validation, it must:

1. Have an active Wazuh agent (or integrated log source).
2. Successfully forward logs to the Wazuh Manager.
3. Appear in the Wazuh dashboard with current heartbeat status.
4. Be time-synchronized with the enterprise NTP source.

Systems not meeting these requirements are considered **monitoring gaps** and must be documented.

2.4 Explicitly Out of Scope

The following activities are not included within this SOP:

- Denial-of-service simulations
- Destructive ransomware execution
- Exposure of real ePHI
- Third-party systems not monitored by Wazuh

These activities require separate authorization and testing procedures.

2.5 Assumptions

This SOP assumes:

- Logging policies are already enabled per baseline configuration.
- Sysmon (Windows) and auditd (Linux) are configured where required.
- FIM policies are deployed to designated sensitive directories.
- Network telemetry sources are properly integrated (if applicable).

3. Compliance Alignment

This validation program is designed to support compliance with the HIPAA Security Rule as defined under **45 CFR Part 164 Subpart C**. The procedures outlined in this SOP validate technical safeguards, administrative controls, and audit mechanisms required to protect electronic Protected Health Information (ePHI).

The validation activities described herein are aligned to the following regulatory requirements.

3.1 45 CFR §164.308 – Administrative Safeguards

164.308(a)(1)(ii)(D) – Information System Activity Review

Requirement:
Covered entities must regularly review records of information system activity, including audit logs, access reports, and security incident tracking reports.

Validation Support Through This SOP:

- Verification of log ingestion from all critical systems
- Validation of alert generation for unauthorized access attempts
- Detection of suspicious process execution
- Confirmation of privilege escalation monitoring
- Review of dashboard visibility and alert completeness

This SOP provides measurable assurance that system activity reviews are technically feasible and operationally effective.

164.308(a)(6) – Security Incident Procedures

Requirement:
Policies and procedures must address the identification and response to suspected or known security incidents.

Validation Support:

- Simulation of defined adversarial behaviors
- Measurement of alert latency (MTTD)
- Validation of incident detection accuracy
- Documentation of false positives and false negatives

These procedures validate that monitoring controls can identify and support response to security incidents involving ePHI.

3.2 45 CFR §164.312 – Technical Safeguards

164.312(a)(1) – Access Control

Requirement:
Implement technical policies and procedures to allow access only to authorized persons or software programs.

Validation Support:

- Privilege escalation simulations
- Administrative group change monitoring
- Authentication failure detection
- Brute-force attempt detection

The validation ensures access control enforcement is actively monitored and violations are detectable.

164.312(b) – Audit Controls

Requirement:
Implement hardware, software, and/or procedural mechanisms to record and examine activity in systems containing or using ePHI.

Validation Support:

- Verification of audit log collection from Windows, Linux, databases, and network devices
- File Integrity Monitoring (FIM) of sensitive directories
- Log tampering detection
- Agent heartbeat monitoring
- Confirmation of log preservation and indexing

This SOP directly validates the effectiveness of Wazuh as an audit control mechanism.

164.312(c)(1) – Integrity

Requirement:
Protect ePHI from improper alteration or destruction.

Validation Support:

- FIM burst detection simulations
- Log modification detection

- Agent tampering detection
- Monitoring of unauthorized file access

These tests ensure integrity monitoring controls are operational and capable of detecting improper modification.

164.312(e)(1) – Transmission Security

Requirement:
Implement technical security measures to guard against unauthorized access to ePHI transmitted over electronic networks.

Validation Support:

- Data exfiltration simulations
- Outbound traffic anomaly detection
- VPN authentication monitoring

This validation confirms monitoring visibility into potential unauthorized data transmission.

3.3 45 CFR §164.306 – Security Standards: General Rules

The HIPAA Security Rule requires covered entities to:

- Ensure confidentiality, integrity, and availability of ePHI
- Protect against reasonably anticipated threats
- Protect against reasonably anticipated impermissible uses or disclosures
- Ensure workforce compliance

This SOP supports these requirements by validating:

- Confidentiality through access and authentication monitoring
- Integrity through FIM and log tampering detection
- Availability through agent health and monitoring continuity
- Threat detection through MITRE ATT&CK-aligned adversarial simulations

3.4 Evidence Generation for Audit Readiness

The procedures in this SOP produce documented evidence that can be retained for compliance review, including:

- Alert screenshots

- Rule IDs and severity classifications
- Detection timestamps
- Mean Time to Detect (MTTD) metrics
- Event correlation summaries
- False positive and false negative analysis
- Change management records for rule tuning

These artifacts provide demonstrable proof of active security monitoring, continuous improvement, and regulatory alignment.

3.5 Risk-Based Approach Alignment

HIPAA requires a risk-based implementation of safeguards. This validation framework:

- Tests high-risk behaviors aligned to real-world attack techniques
- Uses MITRE ATT&CK mapping for adversarial coverage validation
- Measures detection gaps to inform risk remediation
- Supports continuous control improvement

By performing structured adversarial simulations, the organization demonstrates proactive monitoring assurance rather than reactive compliance.

4. Test Execution Methodology

All validation exercises performed under this SOP must follow a standardized, repeatable methodology to ensure consistency, audit defensibility, and measurable outcomes.

This methodology ensures that each simulation produces actionable detection data without impacting system integrity or ePHI confidentiality.

4.1 Pre-Validation Readiness Verification

Prior to executing any simulation, the following conditions must be verified:

1. **Agent Health**
    - All in-scope systems show active heartbeat in the Wazuh Dashboard.
    - No agents are in disconnected or error state.
2. **Log Source Validation**
    - Windows Security logs are forwarding successfully.
    - Sysmon (if deployed) is operational.
    - Linux auditd logs are present and current.
    - Database audit logs (if enabled) are ingesting.
    - Network/VPN logs (if integrated) are visible.
3. **Time Synchronization**
    - All systems are synchronized to the enterprise NTP source.
    - Time drift does not exceed acceptable tolerance (e.g., ±1 minute).
4. **FIM Baseline Integrity**
    - File Integrity Monitoring baseline has been established.
    - No pending FIM database rebuild operations are active.
5. **Change Control Confirmation**
    - Validation window is approved.
    - Impact has been assessed.
    - Stakeholders notified (if required).

Failure to meet readiness criteria must be documented as a monitoring gap.

4.2 Controlled Simulation Execution

Each simulation must follow this structured flow:

Step 1 – Baseline Timestamp Capture

Record:

- Execution start time (UTC)
- System hostname
- User account used
- Simulation identifier

Step 2 – Execute Simulation

- Use only approved test accounts.
- Use only designated test directories.
- Avoid destructive actions.
- Avoid exposure of live ePHI.

Step 3 – Alert Monitoring

- Monitor Wazuh Dashboard in real time.
- Capture:
    - First alert timestamp
    - Rule ID
    - Severity
    - MITRE ATT&CK mapping
    - Source and destination details

Step 4 – Detection Time Calculation

Calculate:

Mean Time to Detect (MTTD) =
Alert Timestamp – Execution Timestamp

Record MTTD in seconds.

4.3 Alert Validation Criteria

Each alert must be reviewed for:

- Accurate severity classification
- Complete contextual enrichment (user, host, IP, file path, process)
- Correct MITRE mapping
- Proper rule correlation (if applicable)
- Dashboard indexing visibility

Alerts lacking required context must be documented as enrichment deficiencies.

4.4 False Positive / False Negative Evaluation

After each simulation:

- Confirm alert was triggered as expected.
- If no alert occurs → classify as False Negative.
- If alert triggers incorrectly or excessively → classify as False Positive.
- Document:
    - Root cause
    - Rule tuning recommendation
    - Configuration gap

This ensures continuous detection improvement.

4.5 Evidence Collection Requirements

For each simulation, collect and retain:

- Screenshot of Wazuh alert
- Raw event log excerpt
- Rule ID and rule description
- MTTD measurement
- Hostname and agent ID
- MITRE technique reference
- Cleanup confirmation

Evidence must be stored in accordance with organizational retention policy and available for compliance review.

4.6 Post-Simulation Cleanup & Validation

After each test:

1. Remove all test artifacts.
2. Confirm test accounts are removed (if created).
3. Verify:
    - Wazuh agents are healthy.
    - No persistent configuration changes remain.
    - No FIM baseline corruption occurred.
4. Record cleanup completion timestamp.

Failure to perform cleanup invalidates test completion.

4.7 Escalation and Remediation Workflow

If critical detection failures are identified:

- Notify Security Operations leadership.
- Open formal detection engineering ticket.
- Perform root cause analysis.
- Implement rule/configuration adjustment.
- Re-test scenario before closing issue.

No validation scenario is considered complete until detection gaps are addressed or formally risk-accepted.

5. Attack Simulation Overview Table

#	Attack Scenario	Control Objective	Primary Log Source	Correlation Required	Expected Wazuh Detection	MITRE ATT&CK	Severity
1	Unauthorized PHI File Access	Confidentiality Monitoring	Windows Event ID 4663	Threshold-based	Excessive file access alert	T1005	High
2	Mass File Modification (Ransomware Simulation)	Integrity Monitoring	FIM Events	Burst correlation	High-volume modification alert	T1486	Critical
3	Privilege Escalation	Access Control Enforcement	Event ID 4728/4732	Single event	Administrative group change alert	T1098	High
4	Brute Force Authentication	Authentication Monitoring	Event ID 4625 / SSH logs	Threshold-based	Brute-force correlation alert	T1110	High
5	Suspicious Process Execution	Endpoint Threat Detection	Sysmon Event ID 1	Rule-based	Suspicious process execution alert	T1059	High
6	Log Tampering	Audit Integrity Protection	Log clear events	Single event	Security log tampering alert	T1070	Critical
7	Data Exfiltration	Transmission Security	Network telemetry	Volume-based correlation	Outbound anomaly alert	T1041	Critical
8	Persistence via Scheduled Task	Persistence Detection	Event ID 4698	Single event	Task creation alert	T1053	High
9	Agent Tampering	Monitoring Continuity	Wazuh Agent Heartbeat	Status monitoring	Agent disconnected alert	T1562	Critical
10	Lateral Movement (SMB)	Lateral Movement Detection	Windows Security Logs	Service creation correlation	Remote service creation alert	T1021	High

6. Detailed Attack Simulation Procedures

All simulations must be executed using designated test accounts and approved test directories. No live ePHI may be modified, encrypted, or transmitted during validation.

Each procedure must document:

- Execution timestamp (UTC)
- Hostname
- Test account used
- Simulation ID reference
- Alert timestamp
- Calculated MTTD

6.1 Unauthorized PHI Access (Insider Threat Simulation)

Objective

Validate detection of excessive access to monitored ePHI directories in support of confidentiality controls.

Preconditions

- Advanced Audit Policy enabled for Object Access
- Directory auditing enabled on monitored path
- Wazuh agent active and forwarding Security logs

Simulation Method

Perform high-volume read-only enumeration of a monitored directory using an authorized test account.

No files may be modified, deleted, or renamed.

Expected Detection

- Windows Security Event ID 4663
- Object name (file path) recorded
- User SID and account name captured
- Threshold-based or excessive-access alert triggered
- Severity classified as High
- MITRE ATT&CK mapping to Data from Local System (T1005)

Pass Criteria

Alert must include:

- Username
- Hostname
- Full file path
- Timestamp
- Rule ID
- Severity classification
- MITRE mapping
- MTTD ≤ defined threshold (per Section 5 matrix)

Failure Conditions

- No alert generated
- Missing file path context
- Incorrect severity classification
- MTTD exceeds defined threshold

6.2 Ransomware Simulation (Mass File Modification)

Objective

Validate File Integrity Monitoring (FIM) and burst correlation detection in support of integrity safeguards.

Preconditions

- FIM enabled on test directory
- Baseline successfully completed
- No active FIM database rebuild

Simulation Method

Perform bulk file content modifications within a designated test directory.
Do not encrypt files or alter production data.

Expected Detection

- High-volume FIM modification events
- Burst correlation rule triggered
- Aggregated alert generated

- Severity classified as Critical
- MITRE ATT&CK mapping to Data Encrypted for Impact (T1486)

Pass Criteria

- Burst threshold exceeded
- Single aggregated alert generated (not excessive alert flooding)
- Correlation visible in dashboard
- MTTD ≤ defined threshold

Failure Conditions

- No burst detection
- Alert flooding without aggregation
- Missing file path metadata

6.3 Privilege Escalation Attempt

Objective

Validate monitoring of unauthorized administrative group membership changes.

Preconditions

- Security event logging enabled
- Test account created for validation
- No production admin accounts modified

Simulation Method

Temporarily add a test account to a privileged group (local or domain).

Expected Detection

- Windows Event ID 4728 or 4732
- Actor account identified
- Target account identified
- Modified group identified
- Severity High or above
- MITRE ATT&CK mapping to Account Manipulation (T1098)

Pass Criteria

Alert clearly identifies:

- Initiating user
- Modified account
- Group name
- Host system
- Rule ID
- MTTD ≤ defined threshold

Test account must be removed immediately after validation.

Failure Conditions

- No alert
- Incomplete user attribution
- Missing group context

6.4 Brute Force Authentication Attempt

Objective

Validate threshold-based correlation for repeated authentication failures.

Preconditions

- Account lockout policy reviewed
- Dedicated test account used
- Logging enabled for failed authentication

Simulation Method

Perform repeated failed login attempts via SSH or RDP within defined threshold limits.

Expected Detection

- Event ID 4625 (Windows) or failed SSH log entries
- Correlation rule triggered after threshold
- MITRE ATT&CK mapping to Brute Force (T1110)
- Severity High

Pass Criteria

Alert includes:

- Source IP
- Target account

- Hostname
- Attempt count
- Time window
- Rule ID
- MTTD ≤ defined threshold

Failure Conditions

- Alert triggers before threshold
- Alert fails to trigger after threshold
- Missing source IP attribution

6.5 Log Tampering Simulation

Objective

Validate detection of audit log manipulation attempts.

Preconditions

- Security log auditing enabled
- Log integrity monitoring active

Simulation Method

Perform a non-destructive log state modification event in a controlled manner.
Full log clearing in production environments is prohibited.

Expected Detection

- Event indicating log clear or tampering attempt
- Severity High or Critical
- Immediate alert generation
- MITRE ATT&CK mapping to Indicator Removal (T1070)

Pass Criteria

- Alert generated within 30 seconds
- Log source identified
- Hostname and actor captured

Failure Conditions

- Delayed detection
- Missing actor attribution

- No alert generated

6.6 Data Exfiltration Simulation

Objective

Validate monitoring of anomalous outbound data transfer patterns.

Preconditions

- Network telemetry integrated (if configured)
- Outbound monitoring thresholds defined

Simulation Method

Generate simulated outbound connection or controlled test data transfer within a non-production environment.
No real PHI may be transmitted.

Expected Detection

- Outbound connection logged
- Volume anomaly correlation triggered (if configured)
- Source and destination IP captured
- Severity High or Critical
- MITRE ATT&CK mapping to Exfiltration Over C2 Channel (T1041)

Pass Criteria

Alert includes:

- Source host
- Destination host/IP
- Transfer volume (if applicable)
- Rule ID
- Severity
- MTTD ≤ defined threshold

Failure Conditions

- No outbound visibility
- No anomaly detection despite threshold exceedance
- Missing network attribution

7. Attack Simulation Commands (Safe & Reversible)

All commands must be executed using designated **test accounts**, within approved test directories, and during scheduled validation windows.
Do not execute directly against production PHI without staging approval.

7.1 Unauthorized PHI File Access (T1005 – High)

Objective

Simulate unauthorized sensitive file enumeration to validate detection coverage aligned with **MITRE ATT&CK T1005 – Data from Local System** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Unauthorized access to sensitive directories
- Excessive file enumeration activity
- Read-only reconnaissance behavior
- High-volume file access events
- Threshold-based alerting for sensitive data access

This simulation performs read-only enumeration only. No file modification, deletion, or exfiltration occurs.

Target Scope

- Windows File Server
- Controlled test directory only
- No production PHI or regulated data
- Access auditing must be enabled on the target directory

Simulation Commands

Step 1 – Create Controlled Test Directory

```
New-Item -ItemType Directory -Path "C:\PHI_TEST" -Force
```

Step 2 – Generate Test Files

```
1..200 | ForEach-Object {
    New-Item -ItemType File -Path "C:\PHI_TEST\testfile$_.txt"
}
```

Step 3 – Simulate Excessive Read-Only Access

```
Get-ChildItem -Path "C:\PHI_TEST" -Recurse -Force
```

Behavioral Simulation

This sequence simulates:

1. Sensitive directory presence
2. Bulk file inventory
3. Read-only reconnaissance
4. High-volume access pattern

This mirrors attacker behavior during:

- Data discovery phase
- Sensitive file reconnaissance
- PHI/PII targeting prior to exfiltration
- Insider threat browsing

No files are opened for modification or altered in any way.

Required Audit Configuration

To generate relevant telemetry:

- Object Access auditing must be enabled
- “Audit File System” success events must be configured
- SACL entries must be applied to the test directory

Without these settings, Event ID 4663 will not be generated.

Expected Windows Telemetry

1. Object Access Events

Log Source:

`Security Log → Microsoft-Windows-Security-Auditing`
Relevant Event ID:

- **4663 – An attempt was made to access an object**

SOC Validation:

- Confirm file path recorded (`C:\PHI_TEST\testfileX.txt`)
- Confirm access type recorded (ReadData/ListDirectory)

- Confirm user account captured
- Confirm access mask details visible
- Confirm access timestamps clustered

2. Access Pattern Characteristics

SOC should observe:

- Multiple 4663 events in short timeframe
- Repeated access to files within same directory
- Consistent subject account
- Enumeration-style access (ListDirectory or ReadAttributes)

The behavioral sequence should resemble:

```
Directory enumeration → High-volume read events → Sensitive path access
```

3. Threshold-Based Alerting

Detection logic should:

- Aggregate multiple file access events
- Trigger alert after defined threshold
- Classify severity as High
- Identify sensitive directory access
- Avoid single-event alert noise

Alerting should be based on:

- X number of file accesses within Y minutes
- Access to regulated or tagged directories
- Non-standard account accessing sensitive path
- Access outside normal working hours (if applicable)

Risk Context

Unauthorized file access is commonly associated with:

- Data discovery prior to exfiltration
- Insider threat activity
- Credential compromise

- Ransomware reconnaissance
- Compliance violations involving PHI/PII

Monitoring read-only enumeration is critical because:

- Exfiltration typically follows discovery
- Early detection reduces impact
- Read-only reconnaissance often precedes encryption or transfer

Expected Detection Outcomes

The SOC should observe:

- Event ID 4663 entries generated
- High-volume file access events
- User account attribution
- Directory path visibility
- Threshold-based alert triggered
- High severity classification (based on rule configuration)

If no alerts are generated, review:

- Object Access audit policy
- SACL configuration on target directory
- Log ingestion completeness
- SIEM correlation thresholds
- Sensitive path tagging logic

Cleanup

```
Remove-Item -Recurse -Force "C:\PHI_TEST"
```

Expected Cleanup Telemetry

SOC should validate:

- Directory deletion event logged
- File deletion events captured (if monitored)
- Cleanup timestamp aligns with simulation window

- No residual test files remain

SOC Validation Checklist

Confirm the following:

- Event ID 4663 generated for file access
- Access type recorded correctly (Read/List)
- User account clearly identified
- File path visible in logs
- High-volume access threshold triggered alert
- Alert severity aligned with policy
- Cleanup events logged

Detection Engineering Considerations

Evaluate whether detection logic includes:

- Sensitive directory monitoring
- Rate-based file access thresholds
- User behavior anomaly detection
- Correlation with authentication events
- Off-hours access detection
- Access to high-risk file shares

If alerting is excessive or insufficient, review:

- Threshold tuning
- Baseline access patterns
- Exclusions for service accounts
- Aggregation logic within SIEM
- Sensitivity classification rules

7.2 Mass File Modification (FIM Burst Simulation) (T1486 – Critical)

Objective

Simulate high-volume file modification activity to validate detection coverage aligned with **MITRE ATT&CK T1486 – Data Encrypted for Impact** under MITRE ATT&CK.

Although this test does not encrypt files, it replicates the behavioral pattern associated with ransomware:

- Rapid file creation
- High-volume file modification
- Burst of file integrity monitoring (FIM) events
- Potential resource spike on file server

This simulation is safe and does not rename, encrypt, or damage production data.

⚠ Execute only within a dedicated test directory.

Target Scope

- File Server
- Dedicated test directory only
- No production shares
- No business-critical data

Windows Simulation

Step 1 – Create Test Directory

```
New-Item -ItemType Directory -Path "C:\FIM_TEST" -Force
```

Step 2 – Create 200 Test Files

```
1..200 | ForEach-Object {
    Set-Content -Path "C:\FIM_TEST\file$_.txt" -Value "Test data $_"
}
```

Step 3 – Modify All Files

```
Get-ChildItem "C:\FIM_TEST\*.txt" | ForEach-Object {
    Add-Content $_.FullName "Modified"
}
```

Behavioral Simulation

This sequence simulates:

1. Bulk file creation
2. Rapid modification of existing files
3. High-frequency write operations
4. Burst-style filesystem activity

This mirrors early-stage ransomware behavior where:

- Files are enumerated
- Files are modified or encrypted
- Changes occur in rapid succession
- Monitoring systems experience event spikes

No encryption or renaming is performed.

Expected Windows Telemetry

1. File Creation Events

SOC should observe:

- Creation of 200 files
- Directory creation event
- Timestamp clustering
- File ownership and creator visibility

If FIM is enabled:

- 200 file creation alerts
- Directory creation alert

2. File Modification Events

SOC should observe:

- 200 modification events
- Rapid modification timestamps
- Content change detection (if hash-based monitoring enabled)

- Updated file hash values

If hash monitoring is configured:

- Hash change events should be generated

3. Burst Detection Behavior

SOC should validate:

- Event rate spike detection
- Aggregation of file modifications into a single correlated alert
- Classification as Critical severity (if ransomware rules enabled)
- Threshold-based trigger (e.g., X file modifications within Y seconds)

The key objective is validating aggregation, not individual alert spam.

Linux Equivalent

Step 1 – Create Test Directory

```
mkdir -p /tmp/fim_test
```

Step 2 – Create 200 Files

```
for i in {1..200}; do echo "test $i" > /tmp/fim_test/file$i.txt; done
```

Step 3 – Modify Files

```
for f in /tmp/fim_test/*.txt; do echo "modified" >> "$f"; done
```

Expected Linux Telemetry

If auditd or FIM monitoring is enabled, SOC should observe:

- Directory creation event
- 200 file creation events
- 200 file modification events
- Timestamp clustering
- User and process attribution
- Parent process visibility (bash)

SOC should verify:

- Write operations logged
- File path visibility

- High-frequency modification detection
- Aggregated alert generation (if configured)

Detection Objectives

The SOC should confirm detection capability for:

- High-volume file modifications within short timeframe
- Multiple write operations to same directory
- Rapid hash changes
- File server activity spikes
- Abnormal file operation patterns

Detection should focus on behavior patterns:

`Bulk file creation → Rapid modification burst → High event volume`
Rather than single-file alerts.

Risk Context

Mass file modification is a core behavioral indicator of ransomware.

Real-world ransomware typically:

- Enumerates accessible directories
- Modifies large numbers of files rapidly
- Changes file extensions
- Alters file content (encryption)
- Drops ransom notes

This simulation validates monitoring logic without causing operational impact.

Expected Detection Outcomes

SOC should observe:

- High-volume FIM events
- Aggregated burst alert
- Critical severity classification
- Clear identification of affected directory

- User and process attribution
- Time-bound clustering

If no alert is generated, review:

- FIM scope and path inclusion
- Event threshold configuration
- SIEM aggregation logic
- Hash monitoring enablement
- File server logging configuration

Cleanup

Windows

```
Remove-Item -Recurse -Force "C:\FIM_TEST"
```

Linux

```
rm -rf /tmp/fim_test
```

Expected Cleanup Telemetry

SOC should validate:

- Directory deletion event logged
- Bulk file deletion events (if monitored)
- Cleanup timestamp aligns with simulation window
- No residual test artifacts remain

SOC Validation Checklist

Confirm the following:

Windows

- Directory creation logged
- 200 file creation events captured
- 200 file modification events captured
- Burst aggregation alert triggered
- Severity classified as Critical

- User and process attribution visible

Linux

- Directory creation logged
- File creation events captured
- File modification events captured
- Aggregation logic triggered
- Alert severity aligned with policy

Detection Engineering Considerations

Evaluate whether detection logic includes:

- Rate-based file modification thresholds
- Directory-level anomaly detection
- Hash change correlation
- Ransomware behavioral rule sets
- Suppression of alert storms in favor of aggregated alerts
- Baseline comparison for file servers

If alerts are not triggered appropriately, review:

- FIM path inclusion scope
- SIEM correlation thresholds
- Agent configuration for real-time monitoring
- Performance tuning for high-volume file events

7.3 Privilege Escalation Attempt (T1098 – High)

Objective

Simulate unauthorized privilege escalation activity to validate detection coverage aligned with **MITRE ATT&CK T1098 – Account Manipulation** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Creation of new local user accounts
- Modification of privileged group membership
- Addition of users to the local Administrators group
- Administrative privilege escalation attempts
- Account lifecycle monitoring

⚠ A temporary, dedicated test account must be used. Cleanup is mandatory after testing.

Simulation Commands

Step 1 – Create Test Account

```
net user wazuh_testuser P@ssw0rd123! /add
```

Behavioral Simulation

This command:

- Creates a new local user account
- Assigns a password
- Registers the account in the local SAM database

This simulates an attacker creating a new account to establish persistence or prepare for privilege escalation.

Expected Telemetry – Account Creation

Log Source:

```
Security Log → Microsoft-Windows-Security-Auditing
```

Relevant Event ID:

- **4720 – A user account was created**

SOC Validation:

- Confirm new account name logged

- Confirm account creator identified
- Confirm timestamp captured
- Confirm target system recorded

Step 2 – Add User to Local Administrators Group

```
net localgroup Administrators wazuh_testuser /add
```

Behavioral Simulation

This command:

- Modifies membership of the local Administrators group
- Grants elevated privileges to the test account
- Simulates privilege escalation via group modification

This reflects common attacker behavior:

- Elevating a low-privileged account
- Maintaining persistence via administrative membership
- Establishing control over the host

Expected Telemetry – Group Membership Modification

Log Source:

```
Security Log → Microsoft-Windows-Security-Auditing
```

Relevant Event IDs (depending on system configuration):

- **4732 – A member was added to a security-enabled local group**
- **4728 – A member was added to a security-enabled global group** (if domain environment)
- **4756 – A member was added to a universal group** (domain environment)

SOC Validation:

- Confirm group name logged (Administrators)
- Confirm added member logged (wazuh_testuser)
- Confirm initiating account captured
- Confirm elevation context visible

Detection Objectives

The SOC should confirm detection capability for:

- Creation of new local accounts
- Modification of privileged group membership
- Non-standard administrative account creation
- Administrative group changes outside approved change windows
- Account creation followed by immediate privilege assignment

High-risk behavioral sequence:

`Account creation → Admin group modification → Elevated access`
Correlation between Event ID 4720 and Event ID 4732 should trigger High-severity alerting.

Risk Context

Account manipulation is frequently used for:

- Privilege escalation
- Persistence establishment
- Backdoor administrative access
- Evasion of detection by using legitimate accounts

Adversaries often:

- Create hidden local admin accounts
- Add existing accounts to privileged groups
- Modify domain group memberships
- Escalate service accounts

Monitoring privileged group changes is critical in preventing full host compromise.

Expected Detection Outcomes

The SOC should observe:

- Event ID 4720 (account creation)
- Event ID 4732 or 4728 (group modification)
- Clear identification of target group (Administrators)

- Clear identification of initiating account
- Alert triggered for administrative group change
- High severity classification

Cleanup (MANDATORY)

Remove elevated privileges:

```
net localgroup Administrators wazuh_testuser /delete
```

Delete test account:

```
net user wazuh_testuser /delete
```

Expected Cleanup Telemetry

Relevant Event IDs:

- **4733 – A member was removed from a security-enabled local group**
- **4726 – A user account was deleted**

SOC Validation:

- Confirm removal from Administrators logged
- Confirm account deletion logged
- Confirm no residual privileged accounts remain
- Confirm cleanup timeline aligns with simulation window

SOC Validation Checklist

Confirm the following:

- Account creation event logged (4720)
- Administrative group modification logged (4732 / 4728)
- Initiating account identified
- Elevated privileges granted visible
- Alert triggered for admin group modification
- Cleanup events logged (4733, 4726)
- No orphaned privileged account remains

Detection Engineering Considerations

Evaluate whether detection logic includes:

- Monitoring of Administrators group changes
- Correlation between account creation and privilege escalation
- Alerting for new admin accounts outside maintenance windows
- Tracking of local vs domain group modifications
- Baseline comparison for privileged group membership

If alerts do not trigger, review:

- Audit policy configuration for account management
- Security log ingestion completeness
- SIEM correlation rules
- Privileged group monitoring coverage

7.4 Brute Force Authentication (T1110 – High)

Objective

Simulate repeated failed authentication attempts to validate detection coverage aligned with **MITRE ATT&CK T1110 – Brute Force** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Repeated authentication failures
- Account lockout conditions
- Threshold-based correlation alerts
- Source-to-target authentication patterns
- Credential abuse indicators

⚠ A dedicated test account must be used. Confirm lockout thresholds before executing to avoid operational impact.

Windows Simulation – SMB Authentication Failures

Command

```
for /l %i in (1,1,20) do net use \\localhost\IPC$ /user:wazuh_testuser
WrongPass123
```

Behavioral Simulation

This command:

- Attempts SMB authentication to localhost
- Uses incorrect credentials
- Repeats 20 times in rapid succession
- Generates authentication failure events

This simulates password brute force attempts over SMB, a common lateral movement vector.

Using `\\localhost\IPC$` ensures:

- No external lateral movement
- Controlled log generation
- Predictable event creation

Expected Windows Telemetry

1. Failed Logon Events

Log Source:

`Security Log → Microsoft-Windows-Security-Auditing`
Relevant Event ID:

- **4625 – An account failed to log on**

SOC Validation:

- Confirm username captured (`wazuh_testuser`)
- Confirm Logon Type recorded (typically Type 3 – Network)
- Confirm source IP (127.0.0.1)
- Confirm failure reason displayed (bad password)
- Confirm timestamp clustering visible

2. Account Lockout (If Threshold Reached)

If account lockout policy is triggered:

Relevant Event ID:

- **4740 – A user account was locked out**

SOC Validation:

- Confirm lockout event logged
- Confirm lockout threshold properly enforced
- Confirm alert triggered for account lockout

3. Threshold-Based Correlation

SOC should verify:

- Alert triggered after defined number of failures
- Correlation across multiple 4625 events
- Source-to-target relationship identified
- Brute force detection rule triggered within expected timeframe

Single failed logons should not trigger high-severity alerts. Correlation logic is critical.

Linux Simulation – SSH Authentication Failures

Command

```
for i in {1..20}; do ssh wazuh_testuser@localhost; done
```

Behavioral Simulation

This command:

- Attempts SSH login repeatedly
- Fails authentication (assuming incorrect password)
- Generates multiple failed login events
- Simulates SSH brute force behavior

Using localhost ensures no external exposure.

Expected Linux Telemetry

1. Authentication Logs

Log Source:

`/var/log/auth.log`

`journalctl`

Typical Log Entries:

- Failed password for user
- Invalid user attempts (if applicable)
- PAM authentication failure entries

SOC Validation:

- Confirm username captured
- Confirm source IP (127.0.0.1)
- Confirm failure timestamps
- Confirm repetition pattern visible

2. Threshold-Based Alerting

SOC should verify:

- Correlation of repeated SSH failures
- Alert generated after defined threshold

- Severity classified appropriately (High)
- Detection of rapid authentication attempts

Detection Objectives

The SOC should confirm detection capability for:

- Multiple failed authentication attempts within defined time window
- Same account targeted repeatedly
- Same source targeting multiple accounts (if tested)
- Account lockout events
- Rapid succession authentication attempts

Detection logic should differentiate between:

- User mistyping password (low frequency)
- Automated brute force attempt (high frequency)
- Distributed attack (multiple sources)

Risk Context

Brute force authentication is commonly used for:

- Initial access
- Lateral movement
- Privilege escalation
- Credential stuffing
- Password spraying

Protocols frequently targeted include:

- SMB
- SSH
- RDP
- WinRM
- Web-based authentication portals

Effective detection requires threshold-based correlation, not single-event alerting.

Expected Detection Outcomes

Windows

- Multiple Event ID 4625 entries
- Possible Event ID 4740 (if lockout occurs)
- Alert triggered after threshold reached
- Source IP and username correlation visible

Linux

- Multiple SSH failed authentication logs
- Correlation-based alert triggered
- Source IP identified
- Account-specific pattern detected

Alert severity should be High when threshold exceeded.

Cleanup

No cleanup required unless account lockout occurs.

If account lockout is triggered in an Active Directory environment:

```
Unlock-ADAccount wazuh_testuser
```

SOC should validate:

- Unlock event logged
- Administrative action captured
- Timeline properly recorded

SOC Validation Checklist

Confirm the following:

Windows

- Event ID 4625 logged for each failed attempt
- Logon type captured
- Failure reason visible
- Correlation rule triggered

- Lockout event (if applicable) detected

Linux

- SSH failed authentication logs present
- Username and source IP captured
- Threshold-based alert triggered
- Alert severity aligns with policy

Detection Engineering Considerations

Evaluate whether detection logic includes:

- Sliding time window correlation
- Threshold-based brute force detection
- Lockout event correlation
- Source IP aggregation
- Account-specific targeting analysis
- Suppression for expected noise (e.g., service accounts)

If alerts do not trigger as expected, review:

- Audit policy configuration
- SSH logging level
- SIEM correlation rules
- Lockout threshold configuration
- Log ingestion completeness

7.5 Suspicious Process Execution (T1059 – High)

Objective

Simulate suspicious command execution patterns to validate detection coverage aligned with **MITRE ATT&CK T1059 – Command and Scripting Interpreter** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Encoded PowerShell execution
- Suspicious command-line arguments
- Living-off-the-land binary (LOLBIN) abuse
- Command-line obfuscation patterns
- Process creation telemetry and parent-child relationships

No external payloads are downloaded or executed.

1. Encoded PowerShell Execution (Benign)

Commands

```
$cmd = "Write-Output 'Test Execution'"
$bytes = [System.Text.Encoding]::Unicode.GetBytes($cmd)
$encoded = [Convert]::ToBase64String($bytes)
powershell.exe -EncodedCommand $encoded
```

Behavioral Simulation

This simulates:

- Obfuscated PowerShell execution
- Base64-encoded command usage
- Common attacker tradecraft to evade simple string-based detection

Although the payload only prints output, the execution method mirrors adversary techniques.

Encoded PowerShell is frequently used to:

- Hide malicious logic
- Evade logging and detection
- Deliver in-memory payloads
- Execute post-exploitation scripts

Expected Telemetry

Process Creation

Expected process:

```
powershell.exe
```

Command-line example:

```
powershell.exe -EncodedCommand <Base64String>
```

SOC Validation:

- Confirm command-line logging enabled
- Confirm full Base64 string captured
- Confirm parent process identified
- Verify Event ID 4688 (if enabled)
- Verify Sysmon Event ID 1 (if deployed)

Detection Considerations

SOC should verify detection logic for:

- `-EncodedCommand` parameter usage
- Long Base64 command-line strings
- PowerShell spawned by unusual parent processes
- PowerShell executed from user-writable directories
- Non-interactive PowerShell invocation

Alert severity should be High due to strong correlation with malicious activity.

2. Suspicious LOLBIN Execution – certutil

Command

```
certutil.exe -urlcache -split -f https://example.com
```

Behavioral Simulation

This simulates:

- Use of certutil as a download utility
- Living-off-the-land binary abuse
- HTTPS retrieval via trusted system binary

No malicious file is executed. The command only performs a benign retrieval attempt.

Adversaries commonly abuse certutil to:

- Download payloads
- Decode Base64 files
- Bypass application allowlisting
- Evade detection by using signed binaries

Expected Telemetry

Process:

`certutil.exe`

Command-line flags of interest:

- `-urlcache`
- `-split`
- `-f`

SOC Validation:

- Confirm process creation logged
- Confirm destination URL captured
- Confirm network connection logged
- Confirm process-network correlation available

Detection Considerations

SOC should verify alerting for:

- certutil making outbound connections
- certutil executed by non-administrative users
- certutil retrieving external content
- Unusual certutil usage outside certificate operations

3. Suspicious LOLBIN Execution – rundll32

Command

```
rundll32.exe shell32.dll,Control_RunDLL
```

Behavioral Simulation

This simulates:

- Use of rundll32 for DLL execution

- Living-off-the-land binary invocation
- Indirect code execution method

Although this invocation is benign, rundll32 is frequently abused to:

- Execute malicious DLLs
- Proxy execution through signed Windows binaries
- Bypass application control
- Execute payloads in memory

Expected Telemetry

Process:

`rundll32.exe`
Command-line example:

`rundll32.exe shell32.dll,Control_RunDLL`
SOC Validation:

- Confirm full command-line captured
- Confirm parent process identified
- Confirm execution context visible
- Verify Sysmon Event ID 1 (if deployed)

Detection Considerations

SOC should verify alerting for:

- rundll32 executing unusual DLL paths
- rundll32 executing from user directories
- rundll32 with suspicious export functions
- rundll32 launched by scripting engines

Expected Detection Outcomes

The SOC should observe:

- Sysmon Event ID 1 (Process Creation), if enabled
- Windows Event ID 4688 (Process Creation), if enabled
- Command-line logging with full parameters

- Potential suspicious command-line alerts
- Network telemetry for certutil outbound connection

Alert classification should align with High severity due to strong overlap with adversary tradecraft.

Optional Cleanup

To remove cached content from certutil:

```
certutil -urlcache * delete
```

SOC should validate:

- Cache deletion logged
- certutil execution recorded
- No unintended files remain in cache directory

SOC Validation Checklist

Confirm the following:

PowerShell

- Encoded command execution logged
- Base64 string visible in telemetry
- Parent process identified
- Alert triggered for encoded command usage

certutil

- Outbound HTTPS connection logged
- URL captured in logs
- Command-line arguments recorded
- Alert triggered for suspicious flags

rundll32

- Full command-line captured
- Execution context visible
- Alert triggered if rule covers rundll32 misuse

Detection Engineering Considerations

Evaluate whether detection logic includes:

- Pattern matching for `-EncodedCommand`
- Base64 length thresholds
- LOLBIN abuse detection rules
- Parent-child process anomaly detection
- Signed binary misuse monitoring
- Process + network correlation logic

If alerts do not trigger, review:

- Command-line logging configuration
- Sysmon deployment and configuration
- Network telemetry integration
- Correlation rules for suspicious binaries

7.6 Log Tampering Simulation (Safe Method) (T1070 – Critical)

Objective

Simulate log tampering behavior in a safe manner to validate detection coverage aligned with **MITRE ATT&CK T1070 – Indicator Removal on Host** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Attempts to disable logging
- Log configuration changes
- Log file metadata modification
- Potential anti-forensic activity
- Tampering targeting security-relevant logs

This simulation does **not** clear or delete logs. It only modifies log state or metadata to generate detectable artifacts without operational impact.

Windows Safe Method

Command

```
wevtutil.exe sl Security /e:false
```

```
wevtutil.exe sl Security /e:true
```

Behavioral Simulation

The `wevtutil sl` (set-log) command modifies the configuration of a Windows Event Log.

- `/e:false` disables the Security log
- `/e:true` re-enables the Security log

This creates log configuration changes without deleting any entries.

This simulates adversary behavior such as:

- Disabling auditing prior to malicious activity
- Temporarily interrupting event generation
- Modifying logging configurations to evade detection

Expected Windows Telemetry

1. Security Log Configuration Changes

Log Source:

`Microsoft-Windows-Eventlog`

`System Log`

Relevant Event IDs may include:

- Event ID 1100 – Event log service shutdown
- Event ID 1101 – Audit events dropped
- Event ID 1102 – Audit log cleared (not expected in this simulation)
- Event ID 4719 – System audit policy changed

SOC Validation:

- Confirm log state change captured
- Confirm initiating user identified
- Confirm command-line logging enabled
- Confirm no log entries were deleted

2. Process Creation Logging

Expected process:

`wevtutil.exe`

SOC Validation:

- Confirm command-line arguments captured
- Confirm execution context (user and privilege level)
- Validate Event ID 4688 (if enabled)
- Validate Sysmon Event ID 1 (if deployed)

Command-line visibility is critical for detecting log tampering attempts.

3. Audit Policy Monitoring

If advanced audit policy monitoring is enabled:

SOC should verify whether:

- Audit configuration change alerts are generated

- Log disable/enable sequence is correlated
- Alert severity is classified as High or Critical

Linux Safe Method

Command

```
touch /var/log/auth.log
```

Behavioral Simulation

The `touch` command updates file metadata (modification timestamp) without altering content.

This simulates:

- Log timestamp manipulation
- Anti-forensic activity attempting to conceal actions
- Unauthorized interaction with authentication logs

No log entries are removed.

Expected Linux Telemetry

1. File Metadata Modification

Log Source (if enabled):

- auditd logs
- File Integrity Monitoring (FIM)
- Endpoint security agent telemetry

SOC Validation:

- Confirm file modification event logged
- Confirm file path recorded (`/var/log/auth.log`)
- Confirm initiating user captured
- Confirm timestamp change detected

2. Command Execution Logging

Expected process:

```
touch
```

SOC Validation:

- Confirm command-line arguments captured

- Confirm parent process recorded
- Confirm user context visible
- Validate auditd EXECVE records (if configured)

Detection Objectives

The SOC should confirm detection capability for:

- Event log disable/enable operations
- Execution of wevtutil targeting Security log
- Modification of critical log files
- Administrative access to authentication logs
- Attempts to manipulate log configuration

Even though no logs are cleared, the behavior should trigger tampering-related monitoring.

Risk Context

Log tampering is commonly used to:

- Remove evidence of compromise
- Disable auditing before privilege escalation
- Conceal lateral movement
- Disrupt forensic investigations

Adversaries may:

- Clear logs
- Disable logging services
- Modify audit policy
- Delete or overwrite log files
- Alter timestamps

This simulation validates detection for early-stage anti-forensic activity without impacting log integrity.

Expected Detection Outcome

The SOC should observe:

Windows

- wevtutil execution logged
- Security log configuration change recorded
- Audit policy change visibility (if configured)
- High/Critical severity alert triggered (if policy enforces)

Linux

- Modification event for `/var/log/auth.log`
- File metadata change recorded
- Command execution logged
- Alert triggered if FIM monitoring enabled

If no alert occurs, review:

- Audit policy configuration
- Event log monitoring coverage
- File Integrity Monitoring scope
- Command-line logging enablement
- Privileged command monitoring rules

Cleanup

No cleanup is required.

All actions are reversible and do not delete or corrupt log entries.

SOC Validation Checklist

Confirm the following:

Windows

- Log disable event recorded
- Log re-enable event recorded
- Initiating account captured
- Command-line arguments logged
- No log entries removed

Linux

- Metadata change detected
- File path correctly identified
- User context captured
- Alert severity appropriate to policy

7.7 Data Exfiltration Simulation (T1041 – Critical)

Objective

Simulate data staging and outbound communication behavior to validate detection coverage aligned with **MITRE ATT&CK T1041 – Exfiltration Over C2 Channel** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Data staging and archive creation
- Large file generation
- Outbound HTTPS connections
- DNS resolution events
- Network egress logging
- Volume-based anomaly detection (if configured)

No real external data transfer is performed. The simulation separates **data preparation** from **network communication** to prevent actual data leakage.

Windows Safe Simulation

Step 1 – Data Staging and Archiving

```
New-Item -ItemType Directory -Path "C:\EXFIL_TEST" -Force
1..100 | ForEach-Object { Set-Content "C:\EXFIL_TEST\data$_.txt" "test" }
Compress-Archive -Path "C:\EXFIL_TEST" -DestinationPath "C:\EXFIL_TEST.zip"
```

Behavioral Simulation

This simulates:

- Bulk file aggregation
- Data staging prior to exfiltration
- Archive preparation for outbound transfer

This reflects common adversary workflow:

1. Collect data
2. Stage locally
3. Compress for efficiency
4. Transfer via C2 channel

Expected Windows Telemetry

1. File Creation Activity

Artifacts created:

- `C:\EXFIL_TEST\` directory
- 100 text files
- `C:\EXFIL_TEST.zip`

SOC Validation:

- Confirm file creation logging enabled
- Confirm archive creation event visibility
- Confirm large file detection (if threshold exists)
- Validate File Integrity Monitoring coverage

2. Process Execution Logging

Expected processes:

- `powershell.exe`
- `Compress-Archive` invocation

SOC Validation:

- Confirm command-line logging enabled
- Confirm parent process context captured
- Verify Event ID 4688 (if enabled)
- Verify Sysmon Event ID 1 (if deployed)

Step 2 – Simulated Outbound Connection

```
Test-NetConnection -ComputerName example.com -Port 443
```

Behavioral Simulation

This command:

- Resolves DNS for example.com
- Initiates TCP connection to port 443
- Performs handshake
- Does not transfer file data

This generates outbound network telemetry without data exfiltration.

Expected Windows Network Telemetry

SOC should observe:

- DNS query for example.com
- Outbound TCP connection to port 443
- Firewall egress log entry
- Proxy log entry (if applicable)
- TLS session metadata (if inspected)

Detection validation:

- Confirm source IP captured
- Confirm destination IP and port logged
- Confirm process associated with connection identified
- Confirm connection duration recorded

Linux Safe Simulation

Step 1 – Data Staging

```
mkdir -p /tmp/exfil_test
```

```
dd if=/dev/zero of=/tmp/exfil_test/dummy.bin bs=1M count=50
```

Behavioral Simulation

This creates a 50 MB file to simulate:

- Bulk data collection
- Large file staging
- Potential DLP threshold testing

SOC Validation:

- Confirm file creation logged
- Confirm file size visible in telemetry
- Validate monitoring of large file creation events

Step 2 – Outbound HTTPS Request (No Upload)

```
curl -I https://example.com
```

The `-I` option performs a HEAD request only.

Behavior:

- DNS resolution
- TLS handshake
- HTTP header retrieval
- No file upload
- No POST request
- No outbound data payload

Expected Linux Network Telemetry

SOC should observe:

- DNS query
- Outbound TCP 443 connection
- TLS session initiation
- curl process execution logged
- HTTP request metadata (if proxy present)

Validation points:

- Confirm process-to-network correlation
- Confirm user context captured
- Confirm egress filtering logging
- Confirm outbound domain logging

Detection Objectives

The SOC should confirm detection capability for:

- Archive creation followed by outbound connection
- Large file staging activity
- HTTPS connections to external domains
- DNS resolution prior to connection
- Suspicious sequence: data staging → outbound network activity

If volume anomaly detection is configured, verify:

- Whether large outbound transfers trigger alerts (note: this simulation does not transfer data)
- Whether file size thresholds generate visibility during staging phase

What This Simulation Does Not Test

- Actual file upload
- DLP content inspection
- POST-based exfiltration detection
- Encrypted payload inspection
- Data loss prevention enforcement

This simulation validates telemetry coverage and logging integrity, not active data leakage prevention.

Cleanup

Windows

```
Remove-Item -Recurse -Force "C:\EXFIL_TEST"
```

```
Remove-Item "C:\EXFIL_TEST.zip"
```

Linux

```
rm -rf /tmp/exfil_test
```

SOC should validate:

- File deletion events logged
- No residual staged data remains
- Cleanup timeline aligns with simulation window

SOC Validation Checklist

Confirm visibility of:

Windows

- Directory creation
- Bulk file generation
- Archive creation
- PowerShell execution logging
- DNS resolution event
- Outbound TCP 443 connection
- Process-network correlation

Linux

- Large file creation event
- curl execution logging
- DNS resolution event
- Outbound TCP 443 connection
- User context visibility

Detection Engineering Considerations

Evaluate whether detection logic triggers on:

- Archive creation followed by outbound HTTPS connection
- Large file staging in temporary directories
- Command-line tools initiating external HTTPS sessions
- Unusual outbound destinations
- Short-duration TLS connections after data staging

If no alerts occur, review:

- DNS logging configuration
- Egress firewall logging coverage
- Proxy visibility
- Endpoint process-to-network telemetry
- File integrity monitoring scope

7.8 Persistence via Scheduled Task (T1053 – High)

Objective

Simulate persistence via Windows Scheduled Task creation to validate logging, alerting, and telemetry coverage aligned with **MITRE ATT&CK T1053 – Scheduled Task/Job** under MITRE ATT&CK.

This test confirms the SOC's ability to detect:

- Scheduled task creation
- SYSTEM-level task execution
- Process creation from Task Scheduler
- Associated file artifact creation
- Task deletion activity

No malicious payload is executed. The command performs a benign file write operation.

Simulation Command

```
schtasks /create /tn "WazuhTestTask" /tr "cmd.exe /c echo test > C:
\temp\wazuh_task.txt" /sc once /st 23:59 /ru SYSTEM /f
```

Parameter Breakdown

Parameter	Description	Detection Relevance
`/create`	Creates a new scheduled task	Triggers task creation events
`/tn "WazuhTestTask"`	Task name	IOC and naming pattern monitoring
`/tr`	Command to execute	Critical for command-line logging
`/sc once`	Run once	Execution timing correlation
`/st 23:59`	Scheduled time	Event timeline validation
`/ru SYSTEM`	Run as Local System	Privileged execution detection
`/f`	Force creation	Overwrites existing task

Behavioral Simulation

This simulates an adversary establishing persistence by:

1. Registering a scheduled task
2. Configuring it to run under SYSTEM
3. Executing a command via Task Scheduler

4. Creating a filesystem artifact
5. Removing persistence (cleanup phase)

The payload used:

```
cmd.exe /c echo test > C:\temp\wazuh_task.txt
```

This ensures observable behavior without risk.

Expected Log Sources and Events

1. Windows Security Log

If “Audit Other Object Access Events” is enabled:

- **Event ID 4698** – Scheduled task created
- **Event ID 4702** – Scheduled task updated (if modified)
- **Event ID 4699** – Scheduled task deleted (during cleanup)

Log source:

```
Microsoft-Windows-Security-Auditing
```

SOC Validation:

- Confirm task name captured
- Confirm run-as account is SYSTEM
- Confirm creator account visibility

2. Task Scheduler Operational Log

Log path:

```
Applications and Services Logs → Microsoft → Windows → TaskScheduler → Operational
```

Expected Events:

- Event ID 106 – Task registered
- Event ID 140 – Task updated
- Event ID 200 – Action started
- Event ID 201 – Action completed

SOC Validation:

- Confirm task execution recorded
- Confirm action command logged

3. Process Creation Telemetry (Critical)

When the task executes:

Parent Process:

`taskeng.exe`
or

`svchost.exe (Schedule service)`
Child Process:

`cmd.exe`
Command Line:

`cmd.exe /c echo test > C:\temp\wazuh_task.txt`
SOC Validation:

- Parent-child relationship visible
- Command-line arguments captured
- Execution context = NT AUTHORITY\SYSTEM
- Event ID 4688 (if enabled)
- Sysmon Event ID 1 (if deployed)

4. File System Artifact

File created:

`C:\temp\wazuh_task.txt`
SOC Validation:

- File creation event logged
- Owner reflects SYSTEM
- File write timestamp aligns with task execution

If File Integrity Monitoring (FIM) is enabled:

- Confirm alert generated for new file creation

Cleanup

```
schtasks /delete /tn "WazuhTestTask" /f
```

```
del C:\temp\wazuh_task.txt
```
Expected Events:

- Event ID 4699 – Scheduled task deleted

- File deletion event (if FIM enabled)
- TaskScheduler Event ID 141 (task removed)

SOC Validation:

- Confirm task removal logged
- Confirm deletion activity captured
- Confirm no orphaned task remains

Detection Engineering Considerations

The SOC should assess whether detections trigger on:

- Task creation with SYSTEM privileges
- Suspicious task names or naming anomalies
- Tasks executing command interpreters (cmd.exe, powershell.exe)
- Task creation followed by immediate execution
- Task creation and deletion within short time window
- Parent process anomalies (non-admin user creating SYSTEM task)

Risk Context

Scheduled tasks are frequently abused because they:

- Provide reliable persistence
- Execute under elevated privileges
- Survive reboots
- Blend into legitimate administrative operations

Persistence via Task Scheduler is commonly observed in ransomware campaigns, commodity loaders, and post-exploitation frameworks.

Validation Checklist for SOC

Confirm the following telemetry is visible:

- Task creation event with full metadata
- SYSTEM execution context
- Command-line arguments captured
- Parent-child process relationship

- File artifact creation
- Task deletion event
- Correlated timeline across logs

If any of these elements are missing, logging or agent coverage gaps should be investigated.

7.9 Agent Tampering Simulation (T1562 – Critical)

Objective

Simulate defensive impairment activity to validate monitoring and alerting aligned with **MITRE ATT&CK T1562 – Impair Defenses** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Security agent service interruption
- Agent restart activity
- Temporary loss of telemetry
- Heartbeat gaps or communication failures
- Service control operations targeting security tooling

This simulation restarts the agent service in a controlled manner. It does **not** terminate processes forcibly or uninstall the agent.

Windows Simulation

```
Restart-Service WazuhSvc
```

Behavioral Impact

- Stops the Wazuh service
- Restarts the service cleanly
- Temporarily interrupts telemetry
- Generates Windows Service Control Manager (SCM) events

This simulates an attacker attempting to disrupt endpoint monitoring without crashing the system.

Linux Simulation

```
systemctl restart wazuh-agent
```

Behavioral Impact

- Stops the wazuh-agent service
- Restarts it via systemd
- Briefly interrupts agent-manager communication
- Generates systemd journal logs

Detection Objectives

The SOC should validate the following:

1. Service State Change Detection

Windows Expected Events:

Log Source:

`System → Service Control Manager`
Relevant Event IDs:

- 7035 – Service control request sent
- 7036 – Service entered stopped state
- 7036 – Service entered running state

SOC Validation:

- Confirm service name logged (WazuhSvc)
- Confirm initiating account visible
- Confirm restart timeline captured

Linux Expected Logs:

Log Source:

`/var/log/syslog`

`journalctl`
SOC Validation:

- Confirm restart logged
- Confirm initiating user recorded (if applicable)
- Confirm restart timestamp aligns with simulation

2. Agent Heartbeat Monitoring

The SOC should verify:

- Temporary heartbeat interruption alert
- Agent disconnect notification
- Reconnection event logged
- Alert severity classification (Critical or High)

Key validation question:
Is there automated alerting when the agent stops communicating for a defined threshold?

3. Telemetry Gap Identification

Restarting the agent creates a short telemetry gap. The SOC should confirm:

- Gap duration is measurable
- Monitoring system logs disconnect event
- Reconnection event is correlated
- No silent failure occurs

4. Service Control Monitoring

SOC detection logic should trigger on:

- Restart of security services
- Stop commands targeting EDR/agent services
- Multiple restart attempts in short timeframe
- Service manipulation by non-administrative users
- Service control executed via suspicious parent processes

Even though this test uses legitimate commands, detection logic should still capture service state transitions.

Risk Context

Adversaries commonly attempt to:

- Stop endpoint agents
- Disable logging services
- Restart agents to interrupt telemetry
- Tamper with defensive tools before deploying payloads

Restarting a service is a lower-noise form of defense evasion compared to:

- Killing the process
- Disabling the service
- Modifying startup type
- Removing the agent

This simulation validates early-stage tampering detection without operational risk.

Expected Detection Outcomes

The SOC should observe:

- Service stop event
- Service start event
- Temporary heartbeat interruption
- Reconnection confirmation
- Alert generated (if monitoring policy configured)

If no alert occurs, detection rules should be reviewed for:

- Service monitoring coverage
- Heartbeat monitoring thresholds
- Agent integrity monitoring
- Service manipulation alerting logic

Cleanup

No cleanup is required.

The agent resumes normal operation after restart.

SOC Validation Checklist

Confirm the following:

- Service stop event logged
- Service start event logged
- Initiating account identified
- Heartbeat interruption detected
- Agent reconnection logged
- Alert severity aligns with policy
- No silent monitoring gap occurred

7.10 Lateral Movement Simulation (T1021 – High)

Objective

Simulate lateral movement activity to validate detection coverage aligned with **MITRE ATT&CK T1021 – Remote Services** under MITRE ATT&CK.

This test evaluates the SOC's ability to detect:

- Remote command execution attempts
- Lateral movement via administrative protocols
- Use of WMI for remote process creation
- SMB authentication activity
- Remote service enumeration behavior

This simulation uses localhost only to prevent real lateral movement while still generating relevant telemetry.

Windows Simulation – WMI Remote Process Execution

```
wmic /node:localhost process call create "cmd.exe /c echo lateral_test > C:
\temp\lat.txt"
```

Behavioral Simulation

This command:

- Uses Windows Management Instrumentation (WMI)
- Targets a remote node (localhost for safety)
- Executes a command remotely
- Creates a file artifact

Although localhost is used, Windows treats this as a remote WMI call, generating remote execution telemetry.

This simulates adversary behavior such as:

- WMI-based lateral movement
- Remote command execution using administrative privileges
- Living-off-the-land remote execution

Expected Windows Telemetry

1. Process Creation Events

Expected child process:

`cmd.exe`
Expected command line:

`cmd.exe /c echo lateral_test > C:\temp\lat.txt`
Parent process:

`wmic.exe`
SOC Validation:

- Confirm parent-child relationship (wmic.exe → cmd.exe)
- Confirm command-line logging enabled
- Confirm execution context (user account used)
- Verify Event ID 4688 (if enabled)
- Verify Sysmon Event ID 1 (if deployed)

2. WMI Activity Logging

If WMI logging is enabled:

Log Source:

`Microsoft-Windows-WMI-Activity/Operational`
Relevant Event IDs:

- 5857 – WMI operation started
- 5858 – WMI operation completed
- 5861 – WMI provider activity

SOC Validation:

- Confirm remote node parameter captured
- Confirm process creation request logged
- Confirm initiating account visibility

3. File Artifact Creation

File created:

`C:\temp\lat.txt`
SOC Validation:

- Confirm file creation event logged

- Confirm timestamp aligns with WMI execution
- Confirm owner and security context
- Validate File Integrity Monitoring coverage (if enabled)

Linux Simulation – SMB Enumeration

```
smbclient -L localhost -U wazuh_testuser
```

Behavioral Simulation

This command:

- Connects to SMB service
- Authenticates using provided user
- Lists available shares
- Does not execute remote commands

This simulates:

- SMB-based reconnaissance
- Credential usage for lateral movement
- Remote service enumeration

No data transfer or remote modification occurs.

Expected Linux Telemetry

1. Authentication Logs

Log Source:

```
/var/log/auth.log
```

```
journalctl
```

SOC Validation:

- Confirm authentication attempt logged
- Confirm username captured (wazuh_testuser)
- Confirm source IP (127.0.0.1)
- Confirm success or failure outcome recorded

2. Samba Logs (if enabled)

Log Source:

```
/var/log/samba/
```

SOC Validation:

- Confirm share enumeration logged
- Confirm session initiation recorded
- Confirm user identity captured

Detection Objectives

The SOC should validate detection capability for:

- WMI-based remote execution attempts
- wmic.exe spawning cmd.exe or powershell.exe
- SMB authentication attempts
- Share enumeration activity
- Remote service usage patterns
- Administrative protocol usage outside baseline behavior

Even though this simulation uses localhost, telemetry should reflect remote service usage patterns.

Risk Context

Lateral movement via remote services is common in:

- Post-compromise internal spread
- Credential abuse
- Ransomware deployment
- Privilege escalation chaining

Common abused protocols include:

- SMB
- WMI
- WinRM
- RDP
- SSH

This simulation specifically validates coverage for WMI and SMB-related activity.

Cleanup

```
del C:\temp\lat.txt
```

SOC should validate:

- File deletion event logged
- File removal timestamp aligns with cleanup
- No persistence artifacts remain

SOC Validation Checklist

Confirm the following telemetry is visible:

Windows

- wmic.exe execution logged
- WMI activity events recorded
- Remote node parameter captured
- cmd.exe spawned by wmic.exe
- File creation logged
- User context visible

Linux

- SMB authentication attempt logged
- Share enumeration recorded
- Username and source IP captured

Detection Engineering Considerations

Evaluate whether alerts trigger on:

- WMI spawning command interpreters
- Remote process creation behavior
- SMB share enumeration from non-standard accounts
- Administrative protocol use outside baseline
- Multiple lateral movement attempts in short timeframe

If no alerts occur, review:

- WMI logging configuration
- Command-line logging coverage
- SMB authentication monitoring
- Correlation rules between authentication and process creation

8. Metrics Collection & Performance Measurement

All validation exercises performed under this SOP must produce measurable detection performance data. Metrics collected under this section serve as the quantitative foundation for compliance validation, detection engineering improvement, and executive reporting.

8.1 Required Metrics Per Simulation

For each simulation scenario, the following data must be recorded:

1. Execution Timestamp (UTC)

- Exact time the simulation action was initiated.
- Source: Test execution log or command output.

2. Detection Timestamp (UTC)

- Time the first corresponding Wazuh alert was generated.
- Source: Wazuh Dashboard alert record.

3. Mean Time to Detect (MTTD)

Formula:

MTTD = Detection Timestamp – Execution Timestamp

Measured in seconds.

MTTD must be compared against the defined target threshold in Section 5.

4. Alert Severity Classification

Record:

- Assigned severity level
- Expected severity level (per validation matrix)
- Variance (if any)

5. Rule Identification

- Rule ID
- Rule description
- Correlation type (single event, threshold, burst, anomaly)

6. False Positive / False Negative Determination

Each simulation must be categorized as:

- **True Positive (TP)** – Alert triggered as expected.
- **False Negative (FN)** – No alert triggered.
- **False Positive (FP)** – Alert triggered incorrectly or excessively.
- **True Negative (TN)** – No alert expected and none triggered (if applicable).

False negatives must trigger remediation workflow per Section 4.

7. Alert Enrichment Completeness

Confirm the alert contains:

- Username
- Hostname
- Source IP (if applicable)
- File path or process name (if applicable)
- MITRE ATT&CK mapping
- Severity level

If fields are missing, document as **Enrichment Deficiency**.

8. Dashboard Visibility Confirmation

Verify:

- Alert appears in Wazuh Dashboard
- Alert is searchable
- Correlated events are linked
- Indexing is functional

8.2 Detection Performance KPIs

The following Key Performance Indicators (KPIs) shall be calculated per validation cycle:

1. Detection Success Rate

Detection Success Rate = (True Positives ÷ Total Simulations) × 100

Target: ≥ 95%

2. False Negative Rate

False Negative Rate = (False Negatives ÷ Total Simulations) × 100

Target: ≤ 5%

3. False Positive Rate

False Positive Rate = (False Positives ÷ Total Simulations) × 100

Target: ≤ 10% (or organization-defined threshold)

4. Average MTTD

Average MTTD = Sum of all MTTD values ÷ Total Simulations

Must remain within defined per-scenario targets.

5. Critical Detection Coverage

Percentage of Critical severity scenarios successfully detected.

Target: 100%

Failure to detect any Critical scenario requires immediate escalation.

8.3 Alert Quality Scoring (Optional but Recommended)

Each alert may be assigned a quality score (1–5 scale) based on:

1. Context completeness
2. Accuracy of severity
3. Clarity of MITRE mapping
4. Correlation effectiveness
5. Actionability

This supports detection engineering maturity tracking.

8.4 Reporting Format

Metrics must be summarized in:

- Tabular format per scenario
- KPI summary dashboard
- Trend comparison against previous validation cycle
- Identification of degraded performance areas

8.5 Escalation Thresholds

Immediate escalation required if:

- Any Critical severity simulation results in False Negative
- MTTD exceeds target by more than 50%
- Agent heartbeat failure is not detected
- Log ingestion gap is identified

9. Reporting Requirements

Upon completion of each validation cycle, a formal Detection Validation Report must be produced. This report serves as compliance evidence, executive communication, and a roadmap for detection engineering improvement.

The report must include the following structured sections.

9.1 Executive Summary

The Executive Summary must provide a high-level overview suitable for leadership and compliance stakeholders.

It shall include:

- Overall Detection Success Rate (%)
- Critical Scenario Detection Status (Pass/Fail)
- Average MTTD
- Number of False Positives
- Number of False Negatives
- High-level risk posture assessment
- Summary of required remediation actions

This section must avoid excessive technical detail and clearly state whether monitoring controls are operating effectively.

9.2 Detection Coverage Matrix

Provide a tabular summary of all tested scenarios including:

- Scenario Name
- Control Objective
- MITRE ATT&CK Technique
- Expected Severity
- Actual Severity
- MTTD
- Detection Result (TP / FP / FN)
- Pass/Fail Status

This matrix provides at-a-glance validation of monitoring effectiveness.

9.3 MITRE ATT&CK Coverage Map

Provide a mapped overview of validated techniques, including:

- Tested techniques (covered)
- Untested techniques (gaps)
- Techniques with detection deficiencies

Where applicable, categorize coverage by:

- Initial Access
- Persistence
- Privilege Escalation
- Defense Evasion
- Credential Access
- Discovery
- Lateral Movement
- Exfiltration
- Impact

This demonstrates adversarial technique visibility maturity.

9.4 Detection Gap Analysis

Document all identified gaps, including:

- False Negatives
- Incomplete alert enrichment
- Incorrect severity classifications
- Excessive MTTD
- Log ingestion failures
- Correlation misconfigurations

For each gap, include:

- Root cause analysis
- Risk impact assessment
- Recommended corrective action
- Target remediation timeline

9.5 False Negative Summary

Provide detailed documentation for each missed detection:

- Scenario ID
- Expected alert behavior
- Observed behavior
- Root cause
- Control impact (Confidentiality / Integrity / Availability)
- Escalation status

Any missed Critical severity detection must be flagged as High Risk.

9.6 False Positive Analysis

Summarize:

- Total false positives
- Source rule(s) generating excessive alerts
- Business impact (alert fatigue, triage burden)
- Recommended rule tuning

Include quantitative data where possible.

9.7 Recommended Rule Tuning & Improvements

Document all proposed improvements, including:

- Rule logic adjustments
- Threshold tuning
- Correlation refinement
- Additional log source integration
- Agent configuration changes
- FIM scope expansion

Each recommendation must include:

- Priority level (Critical / High / Medium / Low)
- Implementation owner
- Change management reference

9.8 Compliance Evidence Package

Attach or reference:

- Alert screenshots
- Raw event samples
- Rule IDs and descriptions
- MTTD calculations
- KPI summary metrics
- Cleanup confirmation logs
- Change management records

Evidence must be retained per organizational retention policy and be retrievable for audit review.

9.9 Overall Control Effectiveness Statement

Conclude the report with one of the following determinations:

- Controls Operating Effectively
- Controls Operating with Minor Deficiencies
- Controls Operating with Significant Deficiencies
- Controls Ineffective

This determination must be supported by quantitative metrics from Section 8.

10. Post-Test Validation & Control Integrity Checklist

Upon completion of all validation simulations, a formal post-test verification must be conducted to ensure monitoring integrity, operational stability, and compliance continuity.

This checklist must be completed before the validation cycle is formally closed.

10.1 Monitoring Integrity Verification

Control Area	Verification Required	Verified (☐/X)	Notes
All Wazuh agents active and reporting	Confirm no agents in disconnected state	☐	
Wazuh Manager operational	No service interruptions or indexing errors	☐	
Wazuh Indexer cluster healthy	No shard or indexing failures	☐	
Wazuh Dashboard accessible	Alerts searchable and visible	☐	
Agent heartbeat monitoring restored	No lingering restart artifacts	☐	

10.2 Logging & Telemetry Validation

Control Area	Verification Required	Verified (☐/X)	Notes
Windows Security logs ingested	Recent event entries visible	☐	
Sysmon logs (if deployed)	Process creation events flowing	☐	
Linux audit logs ingested	auditd entries present	☐	
Database audit logging enabled	Test queries logged	☐	
Network/VPN logs (if integrated)	Current telemetry visible	☐	
Time synchronization intact	No timestamp drift detected	☐	

10.3 File Integrity Monitoring Validation

Control Area	Verification Required	Verified (☐/X)	Notes
FIM enabled on PHI directories	Configuration intact	☐	
FIM baseline integrity confirmed	No unintended baseline rebuild	☐	
Test directories removed	No residual test artifacts	☐	
No production files altered	Confirmed via sampling	☐	

10.4 Alerting & Correlation Validation

Control Area	Verification Required	Verified (☐/X)	Notes
Correlation rules operational	No rule disablement occurred	☐	
Alert escalation functioning	Notification channels verified	☐	
No excessive alert flooding	Post-test noise reviewed	☐	
Severity classifications intact	No rule misconfiguration introduced	☐	

10.5 Cleanup & Account Validation

Control Area	Verification Required	Verified (☐/X)	Notes
Test accounts removed	No lingering elevated accounts	☐	
Temporary directories removed	All test artifacts deleted	☐	
Scheduled tasks removed (if created)	No persistence artifacts remain	☐	
Services restarted (if applicable)	Normal operational state confirmed	☐	

10.6 Compliance Documentation Confirmation

Control Area	Verification Required	Verified (☐/X)	Notes
Metrics recorded	Section 8 completed	☐	
Detection report finalized	Section 9 completed	☐	
Evidence package archived	Screenshots & logs retained	☐	
Change management entries logged	If rule tuning performed	☐	

10.7 Final Control Attestation

The validation cycle is considered complete only when:

- All checklist items are verified.
- No monitoring degradation remains.
- All remediation items are tracked.
- Evidence has been archived per retention policy.

11. Change Control and Review

All modifications to Wazuh rules, correlation logic, agent configuration, logging scope, thresholds, or monitoring architecture resulting from validation activities must follow formal change governance procedures.

No rule tuning may be applied directly to production environments without documented approval and re-validation.

11.1 Change Identification

Changes requiring formal control include:

- Rule severity adjustments
- Threshold modifications
- Correlation logic updates
- New rule creation
- Rule disablement
- FIM scope modifications
- Agent configuration updates
- Log source additions or removals
- Alert escalation configuration changes

All changes must be linked to:

- Validation scenario reference
- Identified detection gap
- False positive or false negative analysis

11.2 Change Documentation Requirements

Each proposed change must include:

- Description of the issue identified
- Root cause analysis
- Risk assessment (Confidentiality / Integrity / Availability impact)
- Proposed technical modification
- Expected improvement outcome
- Rollback plan

Documentation must be recorded in the organization’s formal change management system and assigned a tracking ID.

11.3 Approval Workflow

Before implementation:

1. Change request must be reviewed by:
    - Detection Engineering Lead (if applicable)
    - Security Operations leadership
2. Risk impact must be evaluated.
3. Approval must be formally recorded.

No emergency changes are permitted unless a documented critical detection failure is identified.

11.4 Testing & Validation Requirements

All approved changes must be:

- Implemented first in a staging or lab environment
- Re-tested using the original simulation scenario
- Validated against expected detection criteria
- Measured for new MTTD
- Evaluated for unintended false positives

Production deployment is authorized only after:

- Successful re-validation
- No regression in other detection scenarios
- Documentation of results

11.5 Production Deployment Controls

When deploying to production:

- Deployment window must be approved.
- Monitoring must be heightened during first 24–48 hours.
- Alert noise must be evaluated post-deployment.
- Rollback plan must be available.

Any degradation in detection performance must trigger immediate rollback and review.

11.6 Post-Deployment Review

Within one validation cycle after change deployment:

- Confirm detection improvement occurred.
- Confirm no new false negatives introduced.
- Confirm alert volume remains operationally manageable.

Findings must be documented in the subsequent validation report.

11.7 Audit Traceability

All rule changes must maintain traceability including:

- Change ID
- Approval record
- Validation evidence
- Deployment timestamp
- Responsible engineer

This ensures defensible compliance posture under HIPAA audit review.

11.8 Change Governance Principles

The following principles apply:

- No direct production edits without documentation
- No rule disablement without risk justification
- No severity downgrades without approval
- All detection gaps must have documented resolution path